

Dong Joo Kwon
Professor Chan
DATA_ENG 300-0
23 May 2026

Homework 3: Summary

EC2 Setup Description

For this assignment, I did not create a new EC2 instance and rather used my existing instance. I used the dedicated Python virtual environment “pyspark-venv” that we created in class that is configured with PySpark and Java 11. My homework was completed within a Jupyter Notebook hosted on the instance which I accessed remotely using the server’s returned URL and the provided token.

Dataset Month Used

For this homework, I used the month January of 2026. This isolated month was provided to me through the assignment instructions and I decided to simply use the same subset of data.

Data Cleaning Assumptions

I applied several filtering assumptions when cleaning the data. The combined dataset “combined_df” was generated based off of the following constraints:

- **Missing timestamps** - I removed any records that did not have either a valid pickup or drop-off datetime. This way, all trips are ensured to have complete temporal data. This was done with the following code line: “cleaned_df = combined_df.filter(col("pickup_datetime").isNotNull() & col("dropoff_datetime").isNotNull())”
- **Negative distance** - Any trips that had a recorded distance of zero or less were removed. These must have been canceled rides or system errors that made it into the dataset. This was done with the following code line: “cleaned_df = cleaned_df.filter(col("trip_distance") > 0)”
- **Negative fares** - Any trips that had a negative fare amount were removed in order to maintain financial validity. There is no way a ride could have a negative fare; thus, this must have been a mistake or an error. This was done with the following code line: “cleaned_df = cleaned_df.filter(col("fare_amount") >= 0)”
- **Negative total amount** - Any trips with a negative total cost were excluded, similarly to negative fares. This was done with the following code line: “cleaned_df = cleaned_df.filter(col("total_amount") >= 0)”
- **Cap on trip duration** - Any trips that lasted longer than 24 hours were dropped. These rides were likely errors or rides where the meter was forgotten and was not shut off instead of being realistic passenger trips. This was done with the following code line: “cleaned_df = cleaned_df.filter((col("dropoff_datetime").cast("long") - col("pickup_datetime").cast("long"))) <= 86400)”
- **Ensuring chronological order** - Any records where the drop-off time was recorded as happening before the pickup time were removed, as these are nonsensical records. This was done with the following code line: “cleaned_df = cleaned_df.filter(col("dropoff_datetime") >= col("pickup_datetime"))”

- **Passenger capacity** - I applied a constraint to the passenger count of rides, making the number strictly between 1 and 10, since anything over 10 would be physically impossible vehicle occupancies and anything less than 1 would be non-passenger trips we do not want to consider. This was done with the following code line: “cleaned_df = cleaned_df.filter((col("passenger_count") > 0) & (col("passenger_count") <= 10))”
- **Extreme fares and distances** - I applied an upper bound to the fare amounts and distances. Specifically, any trips with distances over 100 miles or with fares over \$500 were removed. These rides are likely meter or GPS malfunctions. This was done with the following code: “cleaned_df = cleaned_df.filter((col("trip_distance") < 100) & (col("fare_amount") < 500))”
- **Valid location ID** - I filtered out any rides that had a pickup or drop-off location that was not between 1 and 265, since only those are valid NYC Taxi Zones. This was done with the following code: “cleaned_df = cleaned_df.filter(col("PULocationID").between(1, 265) & col("DOLocationID").between(1, 265))”
- **Valid rate codes** - I applied a constraint to the rate code IDs as only being between 1 and 6, since only these values are valid rate code IDs. This was done with the following code line: “cleaned_df = cleaned_df.filter(col("RatecodeID").between(1, 6))”

Answers to Analytics Questions

Question 1: Based on my analysis, the yellow taxi type has significantly more trips at a count of 2442162 whereas the green taxi type has only 33029.

Question 2: The average fare for the yellow taxi type was \$18.70 and the average fare for the green taxi type was \$16.97. Clearly, we can see that the average fare was slightly higher for yellow taxis.

Question 3: The average trip by distance for yellow taxis was 3.17 miles and the average trip by distance for green taxis was 2.62 miles. Clearly, the average distance was longer for yellow.

	Taxi Type	Total Trips	Average Fare	Average Distance
0	yellow	2442162	18.69786	3.171822
1	green	33029	16.97274	2.620311

Question 5: 52.12% of all recorded trips in the cleaned dataset were under 2 miles.

Question 8: I used a Random Forest Model to predict the fare amount. I used non-fare predictor columns, specifically: passenger_count, trip_distance, RatecodeID, PULocationID, and DOLocationID. These columns were chosen because fare amount is likely affected by these factors. The amount paid by the passenger is clearly largely determined by the trip’s distance. The taxi’s meter ticks up based on mileage. The RatecodeID tells the model how the fare is being calculated. A standard metered trip (with code 1) may be calculated differently than a ride with a different code. The PULocationID and DOLocationID, where the trip starts and ends, capture hidden factors of price. Certain routes may be priced similarly, or more congested versus less congested rides may be priced differently. And lastly, the passenger_count may affect the price as well. Larger groups may take vans on longer trips, whereas smaller groups may

have short trips. Regardless of their significance, these are all columns I felt were important to the prediction of the fare amount.

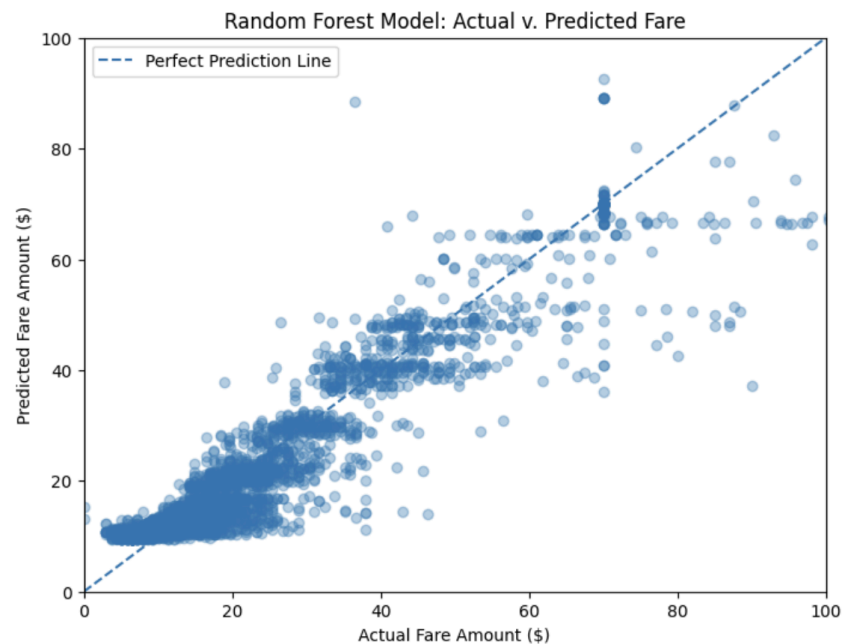
Performance: The model performed pretty consistently across both the testing and training datasets, having a training RMSE of 6.19 and a testing RMSE of 6.28. The very small gap between the two values tells us that the model generalized well to the new data without overfitting.

	Metric	Score
0	Training RMSE	6.19
1	Testing RMSE	6.28

Importance: The trip_distance feature was the most dominant predictor by a very wide margin. It accounted for 65.30% of the model’s decision making. RatecodeID was the second most relevant feature with 23.29%, which logically aligns with NYC taxi pricing rules. Geographic locations, PULocationID and DOLocationID had a minor impact on the fare. The passenger_count proved to be virtually irrelevant to predicting the fare amount. This makes sense since taxis charge by distance and time, not per person.

	Predictor Variable	Importance Score
1	trip_distance	0.6530
2	RatecodeID	0.2329
3	PULocationID	0.0914
4	DOLocationID	0.0226
0	passenger_count	0.0001

Plot: The generated scatter plot visually confirms the model’s accuracy. The bulk of the predictions lie closely along the diagonal “Perfect Prediction Line”, especially for standard trips under \$40. Above \$40 fares, the data points become more scattered and there is slight variance. There are also a few visible horizontal clusters. Ultimately, the model successfully captures the broader pricing trends using only non-fare data.



AI Usage

I used Gemini to help determine how to start my homework. The instructions mentioned an EC2 instance, but we had already set up a PySpark virtual environment in class. I was confused on whether we had to set up a brand new instance, or if I could use my existing one. Ultimately, I confirmed my thoughts that I just had to start my existing EC2 instance, activate the pyspark-venv environment we built in class, and launch my Jupyter Notebook.

I used Gemini to help identify additional data cleaning methods to apply to the RatecodeID, PULocationID, and DOLocationID columns. Looking at their names, since they were “ID”s, I assumed there must be some minimum and maximum ID value these columns could hold, and any others would be nonsensical. So, Gemini helped me determine that there are indeed bounds for these columns. RatecodeIDs are restricted to 1-6 and LocationIDs are restricted to 1-265. I adapted the PySpark filter() logic myself, but used Gemini to identify what these bounds should be.

S3 Output Paths

s3://dongjoo-26spring/taxi_summary_stats.csv
s3://dongjoo-26spring/model_importances_of_predictors.csv
s3://dongjoo-26spring/fare_predictions.png